Practical 1: Student Information Management System

Objective

To understand Python variables, data types, input/output functions, and formatted printing by developing a simple student information system.

Problem Statement

Develop a Python program that collects the details of a student, including Enrollment Number, Name, Branch, Semester, Date of Birth, Mobile Number, and CGPA. The program should display all information in a neatly formatted report similar to a college student profile.

Functional Requirements

- Accept all details from the user.
- Store data using appropriate variables.
- Display the information using formatted output.
- Calculate whether the student is eligible for placement based on CGPA (for example, CGPA $\geq$ 6.5).

INPUT CODE:

```
print("ENROLLMENT NO. : 250283152004")
print("----- STUDENT INFORMATION SYSTEM -----")

enrollment_no = input("Enter Enrollment Number: ")
name = input("Enter Student Name: ")
branch = input("Enter Branch: ")
semester = int(input("Enter Semester: "))
dob = input("Enter Date of Birth (DD-MM-YYYY): ")
mobile = input("Enter Mobile Number: ")
cgpa = float(input("Enter CGPA: "))

if cgpa >= 6.5:
    eligible = "Yes"
else:
    eligible = "No"

print("\n" + "=" * 40)
print("          STUDENT PROFILE REPORT")
print("=" * 40)
print("{:<20}: {}".format("Enrollment No", enrollment_no))
print("{:<20}: {}".format("Name", name))
print("{:<20}: {}".format("Branch", branch))
print("{:<20}: {}".format("Semester", semester))
print("{:<20}: {}".format("Date of Birth", dob))
print("{:<20}: {}".format("Mobile Number", mobile))
print("{:<20}: {:.2f}".format("CGPA", cgpa))
print("{:<20}: {}".format("Placement Eligible", eligible))
print("=" * 40)
```

OUTPUT CODE:

```
ENROLLMENT NO. : 250283152004
----- STUDENT INFORMATION SYSTEM -----
Enter Enrollment Number: 004
Enter Student Name: jay
Enter Branch: aiml
Enter Semester: 5
Enter Date of Birth (DD-MM-YYYY): 06062007
Enter Mobile Number: 45045459
Enter CGPA: 5

========================================
          STUDENT PROFILE REPORT
========================================
Enrollment No       : 004
Name                : jay
Branch              : aiml
Semester            : 5
Date of Birth       : 06062007
Mobile Number       : 45045459
CGPA                : 5.00
Placement Eligible  : No
========================================
```

Practical 2: Electricity Bill Calculator

Objective To learn conditional statements by implementing a real-world electricity billing application. **Problem Statement**
Develop a Python program that calculates the monthly electricity bill based on the number of units consumed. Different tariff rates should be applied according to predefined slabs. The final bill should also include fixed charges and GST.

Sample Tariff

Units	Rate (₹/Unit)
0–100	3.50
101–300	5.00
Above 300	7.50

Fixed Charge = ₹100

GST = 5%

Functional Requirements

- Accept customer name and units consumed.
- Calculate bill according to slab rates.
- Add fixed charges and GST.
- Display a professional electricity bill.

INPUT CODE:

```
print("ENROLLMENT NO. : 250283152004")
print("----- ELECTRICITY BILL CALCULATOR -----")

name = input("Enter Customer Name: ")
units = float(input("Enter Units Consumed: "))

FIXED_CHARGE = 100
GST_RATE = 0.05

if units <= 100:
    bill = units * 3.50
elif units <= 300:
    bill = (100 * 3.50) + (units - 100) * 5.00
else:
    bill = (100 * 3.50) + (200 * 5.00) + (units - 300) * 7.50

gst_amount = bill * GST_RATE
total_bill = bill + FIXED_CHARGE + gst_amount

print("\n" + "=" * 40)
print("           ELECTRICITY BILL")
print("=" * 40)
print("{:<20}: {}".format("Customer Name", name))
print("{:<20}: {}".format("Units Consumed", units))
print("{:<20}: Rs.{:.2f}".format("Energy Charges", bill))
print("{:<20}: Rs.{:.2f}".format("Fixed Charges", FIXED_CHARGE))
print("{:<20}: Rs.{:.2f}".format("GST (5%)", gst_amount))
print("-" * 40)
print("{:<20}: Rs.{:.2f}".format("Total Bill", total_bill))
print("=" * 40)
```

OUTPUT CODE:

```
ENROLLMENT NO. : 250283152004
----- ELECTRICITY BILL CALCULATOR -----
Enter Customer Name: jay
Enter Units Consumed: 50

=======================================
          ELECTRICITY BILL
=======================================
Customer Name        : jay
Units Consumed       : 50.0
Energy Charges       : Rs.175.00
Fixed Charges        : Rs.100.00
GST (5%)             : Rs.8.75
---------------------------------------
Total Bill           : Rs.283.75
=======================================
```

Practical 3: Student Grade Analyzer

Objective

To understand loops, conditional statements, and mathematical operations using an academic result processing system.

Problem Statement

Develop a program that accepts marks obtained by a student in five subjects. The program should calculate the total marks, percentage, grade, and pass/fail status. Display the complete result in a formatted report card.

Grade Policy (Example)

Percentage	Grade
≥90	A+
80–89	A
70–79	B
60–69	C
50–59	D
Below 50	Fail

Functional Requirements

- Accept marks of five subjects.
- Validate that marks are between 0 and 100.
- Calculate total and percentage.
- Assign grade.
- Display pass/fail status.
- Identify the highest and lowest scoring subjects.

INPUT CODE:

```
print("ENROLLMENT NO. : 250283152004")
print("----- STUDENT GRADE ANALYZER -----")

subjects = []
marks = []

for i in range(1, 6):
    subject_name = input("Enter name of Subject " + str(i) + ": ")
    while True:
        mark = float(input("Enter marks obtained in " + subject_name + " (0-100): "))
        if 0 <= mark <= 100:
            break
        else:
            print("Invalid marks! Please enter a value between 0 and 100.")
    subjects.append(subject_name)
    marks.append(mark)

total = sum(marks)
percentage = total / 5

if percentage >= 90:
    grade = "A+"
elif percentage >= 80:
    grade = "A"
elif percentage >= 70:
    grade = "B"
elif percentage >= 60:
    grade = "C"
elif percentage >= 50:
    grade = "D"
else:
    grade = "Fail"

if percentage >= 50:
    status = "Pass"
else:
    status = "Fail"

highest_index = marks.index(max(marks))
```

```
lowest_index = marks.index(min(marks))


print("\n" + "=" * 40)
print("          REPORT CARD")
print("=" * 40)
for i in range(5):
    print("{:<15}: {}".format(subjects[i], marks[i]))
print("-" * 40)
print("{:<15}: {}/500".format("Total Marks", total))
print("{:<15}: {:.2f}%".format("Percentage", percentage))
print("{:<15}: {}".format("Grade", grade))
print("{:<15}: {}".format("Status", status))
print("{:<15}: {} ({})".format("Highest Scoring", subjects[highest_index], marks[highest_index]))
print("{:<15}: {} ({})".format("Lowest Scoring", subjects[lowest_index], marks[lowest_index]))
print("=" * 40)
```

OUTPUT CODE:

```
ENROLLMENT NO. : 250283152004
----- STUDENT GRADE ANALYZER -----
Enter name of Subject 1: math
Enter marks obtained in math (0-100): 57
Enter name of Subject 2: eng
Enter marks obtained in eng (0-100): 80
Enter name of Subject 3: sci
Enter marks obtained in sci (0-100): 28
Enter name of Subject 4: ml
Enter marks obtained in ml (0-100): 90
Enter name of Subject 5: ai
Enter marks obtained in ai (0-100): 45

========================================
          REPORT CARD
========================================
math            : 57.0
eng             : 80.0
sci             : 28.0
ml              : 90.0
ai              : 45.0
----------------------------------------
Total Marks     : 300.0/500
Percentage      : 60.00%
Grade           : C
Status          : Pass
Highest Scoring: ml (90.0)
Lowest Scoring : sci (28.0)
========================================
```

Practical 4: Inventory Management System using Lists and Dictionaries

Objective

To understand Python's mutable data structures by implementing a simple inventory management application.

Problem Statement

Develop an inventory management system for a small stationery shop. Each product should contain Product ID, Product Name, Quantity Available, and Price. The system should allow users to manage inventory records.

Functional Requirements

The program should provide a menu-driven interface to:

- Add a new product
- Search a product using Product ID
- Update product quantity
- Delete a product
- Display all products
- Calculate total inventory value

Inventory Value = Quantity × Price

INPUT CODE:

```
print("ENROLLMENT NO. : 250283152004")
inventory = []

def add_product():
    product = {
        "id": input("Enter Product ID: "),
        "name": input("Enter Product Name: "),
        "quantity": int(input("Enter Quantity: ")),
        "price": float(input("Enter Price: "))
    }
    inventory.append(product)
    print("Product added successfully!\n")

def search_product():
    pid = input("Enter Product ID to search: ")
    for product in inventory:
        if product["id"] == pid:
            print("\nFound: " + str(product) + "\n")
            return
    print("Product not found!\n")

def update_quantity():
    pid = input("Enter Product ID to update: ")
    for product in inventory:
        if product["id"] == pid:
            product["quantity"] = int(input("Enter new quantity: "))
            print("Quantity updated!\n")
            return
    print("Product not found!\n")

def delete_product():
    pid = input("Enter Product ID to delete: ")
    for product in inventory:
        if product["id"] == pid:
            inventory.remove(product)
            print("Product deleted!\n")
            return
    print("Product not found!\n")
```

```
def display_products():
    if not inventory:
        print("Inventory is empty.\n")
        return
    print("\n" + "-" * 55)
    print("{:<10}{:<20}{:<10}{:<10}".format("ID", "Name", "Qty", "Price"))
    print("-" * 55)
    for product in inventory:
        print("{:<10}{:<20}{:<10}{:<10}".format(product["id"], product["name"], product["quantity"], product["price"]))
    print("-" * 55 + "\n")

def total_inventory_value():
    total = sum(p["quantity"] * p["price"] for p in inventory)
    print("\nTotal Inventory Value: Rs.{:.2f}\n".format(total))

while True:
    print("===== INVENTORY MANAGEMENT SYSTEM =====")
    print("1. Add Product")
    print("2. Search Product")
    print("3. Update Quantity")
    print("4. Delete Product")
    print("5. Display Products")
    print("6. Total Inventory Value")
    print("7. Exit")

    choice = input("Enter your choice (1-7): ")
```

```python
if choice == '1':
    add_product()
elif choice == '2':
    search_product()
elif choice == '3':
    update_quantity()
elif choice == '4':
    delete_product()
elif choice == '5':
    display_products()
elif choice == '6':
    total_inventory_value()
elif choice == '7':
    print("Exiting program.")
    break
else:
    print("Invalid choice! Please choose between 1 and 7.\n")
```

OUTPUT CODE

```
ENROLLMENT NO. : 250283152004
===== INVENTORY MANAGEMENT SYSTEM =====
1. Add Product
2. Search Product
3. Update Quantity
4. Delete Product
5. Display Products
6. Total Inventory Value
7. Exit
Product added successfully!

===== INVENTORY MANAGEMENT SYSTEM =====
1. Add Product
2. Search Product
3. Update Quantity
4. Delete Product
5. Display Products
6. Total Inventory Value
7. Exit

--------------------------------------------------------
ID        Name                Qty       Price
--------------------------------------------------------
69        laptop              1         70000.0
--------------------------------------------------------

===== INVENTORY MANAGEMENT SYSTEM =====
1. Add Product
2. Search Product
3. Update Quantity
4. Delete Product
5. Display Products
```

Practical 5: Online Shopping Cart Simulation

Objective

To understand lists, dictionaries, loops, and decision-making by simulating an online shopping cart.

Problem Statement

Develop a shopping cart application similar to an e-commerce website. Users should be able to browse available products, add items to their cart, remove items, and generate the final invoice.

Functional Requirements

- Display available products.
- Add products to the cart.
- Remove products from the cart.
- Modify quantities.
- Calculate subtotal.
- Apply discount:
    - 10% discount for purchases above ₹5000.
- Calculate GST (18%).
- Generate the final bill.

Sample Output

```
Product Qty Price
--------------------------------
Laptop 1 45000
Mouse 2 1200

Subtotal ₹47400
Discount  ₹4740
GST ₹7680
--------------------------------
Total Amount ₹50340
```

INPUT CODE:

```
print("ENROLLMENT NO. : 250283152004")

products = {
    "Laptop": 45000,
    "Mouse": 1200,
    "Keyboard": 800,
    "Monitor": 9000,
    "Headphones": 1500
}

cart = {}

def show_products():
    print("\n----- AVAILABLE PRODUCTS -----")
    for name, price in products.items():
        print("{:<15} Rs.{}".format(name, price))
    print()

def add_to_cart():
    name = input("Enter product name to add: ")
    if name in products:
        qty = int(input("Enter quantity: "))
        cart[name] = cart.get(name, 0) + qty
        print(name + " added to cart!\n")
    else:
        print("Product not available!\n")

def remove_from_cart():
    name = input("Enter product name to remove: ")
    if name in cart:
        del cart[name]
        print(name + " removed from cart!\n")
    else:
        print("Item not found in cart!\n")
```

```python
def modify_quantity():
    name = input("Enter product name to modify: ")
    if name in cart:
        cart[name] = int(input("Enter new quantity: "))
        print("Quantity updated!\n")
    else:
        print("Item not found in cart!\n")

def generate_bill():
    if not cart:
        print("Cart is empty!\n")
        return

    print("\n" + "-" * 40)
    print("{:<15}{:<10}{:<10}".format("Product", "Qty", "Price"))
    print("-" * 40)

    subtotal = 0
    for name, qty in cart.items():
        price = products[name]
        item_total = price * qty
        subtotal += item_total
        print("{:<15}{:<10}Rs.{:<10}".format(name, qty, item_total))

    if subtotal > 5000:
        discount = subtotal * 0.10
    else:
        discount = 0
    after_discount = subtotal - discount
    gst = after_discount * 0.18
    total = after_discount + gst

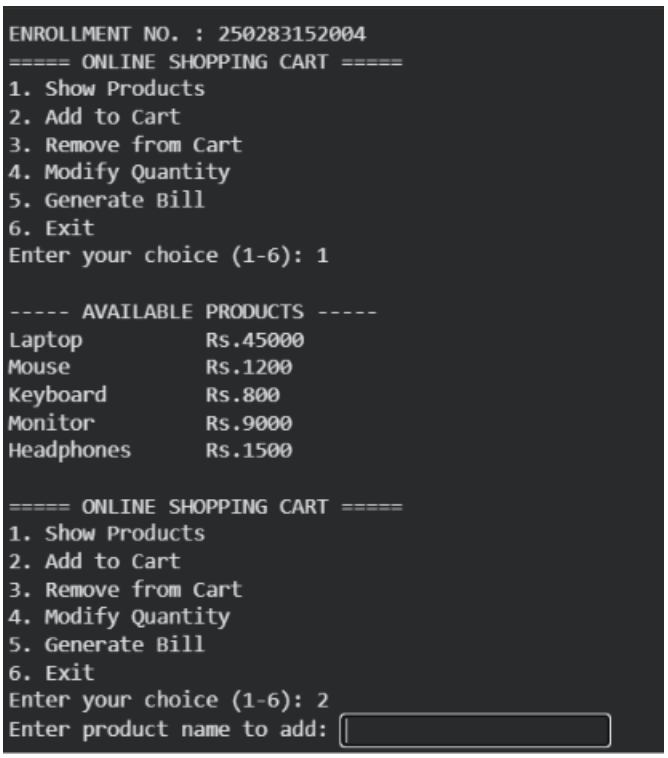
    print("-" * 40)
    print("{:<25}Rs.{:.2f}".format("Subtotal", subtotal))
    print("{:<25}Rs.{:.2f}".format("Discount", discount))
    print("{:<25}Rs.{:.2f}".format("GST (18%)", gst))
    print("-" * 40)
    print("{:<25}Rs.{:.2f}".format("Total Amount", total))
    print("-" * 40 + "\n")
```

```python
while True:
    print("===== ONLINE SHOPPING CART =====")
    print("1. Show Products")
    print("2. Add to Cart")
    print("3. Remove from Cart")
    print("4. Modify Quantity")
    print("5. Generate Bill")
    print("6. Exit")

    choice = input("Enter your choice (1-6): ")

    if choice == "1":
        show_products()
    elif choice == "2":
        add_to_cart()
    elif choice == "3":
        remove_from_cart()
    elif choice == "4":
        modify_quantity()
    elif choice == "5":
        generate_bill()
    elif choice == "6":
        print("Thank you for shopping!")
        break
    else:
        print("Invalid choice! Try again.\n")
```

OUTPUT CODE:


ENROLLMENT NO. : 250283152004
===== ONLINE SHOPPING CART =====
1. Show Products
2. Add to Cart
3. Remove from Cart
4. Modify Quantity
5. Generate Bill
6. Exit
Enter your choice (1-6): 1
----- AVAILABLE PRODUCTS -----
Laptop Rs.45000
Mouse Rs.1200
Keyboard Rs.800
Monitor Rs.9000
Headphones Rs.1500
===== ONLINE SHOPPING CART =====
1. Show Products
2. Add to Cart
3. Remove from Cart
4. Modify Quantity
5. Generate Bill
6. Exit
Enter your choice (1-6): 2
Enter product name to add:

Practical 6: Password Strength Validator

Objective

To understand string manipulation and conditional logic by implementing a password validation system.

Problem Statement

Develop a Python program that checks whether a password satisfies basic cybersecurity requirements before allowing a user to register.

Password Policy

A valid password must contain:

- Minimum 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one digit
- At least one special character (@, #, $, %, &, *)

Functional Requirements

- Accept password from the user.
- Check each validation rule.
- Display specific messages indicating which criteria are not satisfied.
- Display "Strong Password" only if all conditions are met.

Example Output

Enter Password:
Python@123
Password Strength: Strong

or

Enter Password:
python123

Missing:

INPUT CODE:

```
def check_password_strength(password):
    special_chars = "@#$%&*"

    has_upper = any(char.isupper() for char in password)
    has_lower = any(char.islower() for char in password)
    has_digit = any(char.isdigit() for char in password)
    has_special = any(char in special_chars for char in password)
    has_min_length = len(password) >= 8

    missing = []
    if not has_min_length:
        missing.append("Minimum 8 Characters")
    if not has_upper:
        missing.append("Uppercase Letter")
    if not has_lower:
        missing.append("Lowercase Letter")
    if not has_digit:
        missing.append("Digit")
    if not has_special:
        missing.append("Special Character (@,#,$,%,&,*)")

    return missing

password = input("Enter Password: ")
missing_criteria = check_password_strength(password)

if not missing_criteria:
    print("\nPassword Strength: Strong")
else:
    print("\nMissing:")
    for item in missing_criteria:
        print("X " + item)
    print("\nPassword Strength: Weak")
```

OUTPUT CODE:

```
Enter Password: JAY jethava

Missing:
X Digit
X Special Character (@,#,$,%,&,*)

Password Strength: Weak
```